

Netflix Recommendation System

Personalized Content Discovery using Matrix Factorization

Project Report

Team Member 1: Rishabh Gupta(24114077)

Team Member 2: Adi Jain(24114003)

Department: Computer Science and Engineering

Institution: IIT Roorkee

Submitted on: June 12, 2026

Abstract

Recommendation systems are widely used in modern digital platforms to help users discover relevant content from large collections of available items. Streaming platforms such as Netflix use recommendation algorithms to personalize user experience, reduce information overload, and improve user engagement.

This project presents the design and implementation of a Personalized Content Discovery and Recommendation System using the Netflix Prize Dataset. The system uses collaborative filtering and matrix factorization techniques to learn hidden user preferences and movie characteristics from historical rating data. Instead of building only an offline machine learning model, this project implements an end-to-end machine learning engineering pipeline consisting of data preprocessing, sampling, model training, evaluation, experiment tracking, API serving, and interactive dashboard visualization.

The implemented system uses Polars and PyArrow for efficient large-scale data processing, Surprise SVD for explicit rating prediction, Implicit ALS for scalable matrix factorization, MLflow for experiment tracking, FastAPI for serving recommendations, and Streamlit for building a user-facing dashboard. The SVD model achieved an RMSE of 0.8483 and an MAE of 0.6566, showing that the model can effectively predict user preferences on sparse user-movie interaction data.

Keywords: Recommendation System, Netflix Prize Dataset, Collaborative Filtering, Matrix Factorization, SVD, ALS, MLflow, FastAPI, Streamlit.

Contents

Abstract

1	Introduction	1
2	Motivation	2
3	Objectives	2
4	Literature Review	2
5	Collaborative Filtering	2
5.1	User-Based Collaborative Filtering	3
5.2	Item-Based Collaborative Filtering	3
6	Matrix Factorization	3
7	Singular Value Decomposition	3
8	Alternating Least Squares	3
9	Reason for Choosing SVD and ALS	3
10	Problem Statement	4
11	Main Challenges	4

12 Proposed Solution	4
13 Dataset Description and Preprocessing	5
14 Netflix Prize Dataset	5
15 Raw Data Format	5
16 Processed Dataset Schema	5
17 Why Polars and Parquet Were Used	6
18 Sampling Strategy	6
19 Proposed Methodology	6
20 Workflow Overview	7
21 How the Recommendation Problem is Solved	7
22 Recommendation Generation	8
23 System Architecture	8
24 Architecture Overview	8
25 Technology Stack	9
26 Model Development	9
27 SVD Model	9
28 ALS Model	9
29 Model Serialization	10
30 Experimental Results	10
31 Evaluation Metrics	10
31.1 Root Mean Squared Error	10
31.2 Mean Absolute Error	10
32 SVD Results	10
33 Model Comparison	11
34 Experiment Tracking with MLflow	11
35 Need for Experiment Tracking	11
36 Tracked Parameters	11

37 Tracked Metrics	11
38 Tracked Artifacts	12
39 Backend API and User Interface	12
40 FastAPI Backend	12
40.1 Health Check Endpoint	12
40.2 Recommendation Endpoint	12
41 Streamlit Dashboard	12
42 Frontend-Backend Flow	13
43 System Validation	13
44 Results and Discussion	13
45 Completed Components	14
46 Discussion	14
47 Limitations and Future Work	14
48 Limitations	14
49 Future Work	14
50 Conclusion	15

1 Introduction

The amount of digital content available today has increased rapidly. Users of online platforms are often presented with thousands or millions of available choices. In such environments, manually searching for relevant content becomes difficult and inefficient. Recommendation systems solve this problem by automatically suggesting items that are likely to match the interests of individual users. Recommendation systems are used in several real-world platforms such as Netflix, YouTube, Amazon, Spotify, and social media applications. These systems improve user experience by presenting personalized content instead of showing the same results to every user. In the context of movie streaming, recommendations help users discover movies they are likely to enjoy based on their previous ratings and interaction history.

This project focuses on building a personalized movie recommendation system using the Netflix Prize Dataset. The dataset contains a large number of user-movie rating interactions and represents a realistic sparse recommendation problem. The main idea is to learn hidden patterns from historical ratings and use those patterns to recommend unseen movies to users.

The project was developed in phases. First, the raw dataset was parsed and converted into an efficient Parquet format. Then, sampled datasets were created to allow faster experimentation on laptop hardware. After that, matrix factorization models were trained and evaluated. Finally, the trained

recommendation model was connected to a FastAPI backend and displayed through a Streamlit dashboard.

2 Motivation

The motivation behind this project is to build a recommendation system that goes beyond a simple notebook-based machine learning model. Many academic projects stop after training a model, but real-world machine learning systems require additional engineering components such as data pipelines, model serialization, API endpoints, experiment tracking, and interactive interfaces.

This project demonstrates the complete lifecycle of a recommendation system, starting from raw data and ending with a working web application.

3 Objectives

The major objectives of this project are:

- To process the Netflix Prize Dataset efficiently using modern data processing tools.
- To convert raw text-based rating files into structured Parquet files.
- To create smaller representative samples for development and experimentation.
- To implement matrix factorization-based recommendation models.
- To evaluate model performance using RMSE and MAE.
- To track model experiments using MLflow.
- To serve recommendations using FastAPI.
- To create an interactive Streamlit dashboard for user-facing recommendations.

4 Literature Review

Recommendation systems are generally classified into collaborative filtering, content-based filtering, and hybrid recommendation systems. This project mainly focuses on collaborative filtering because the Netflix Prize Dataset contains user-movie rating interactions.

5 Collaborative Filtering

Collaborative filtering is based on the assumption that users who behaved similarly in the past are likely to behave similarly in the future. It uses historical user-item interactions to recommend new items.

There are two traditional forms of collaborative filtering:

5.1 User-Based Collaborative Filtering

User-based collaborative filtering finds users with similar rating patterns and recommends items liked by similar users. However, this approach does not scale well when the number of users is very large. For a dataset with hundreds of thousands of users, computing user-user similarity becomes computationally expensive.

5.2 Item-Based Collaborative Filtering

Item-based collaborative filtering computes similarity between items and recommends items similar to those already liked by the user. It is generally more scalable than user-based collaborative filtering but still requires maintaining item similarity matrices.

6 Matrix Factorization

Matrix factorization is a more scalable and powerful approach for recommendation systems. It represents users and items in a lower-dimensional latent factor space. Each user and movie is represented by a vector of hidden features. The predicted rating is calculated using the interaction between the user vector and the movie vector.

Matrix factorization is especially effective for sparse datasets because it can infer hidden relationships even when most user-item interactions are missing.

7 Singular Value Decomposition

Singular Value Decomposition decomposes a rating matrix into latent user and item factor matrices. In recommendation systems, SVD learns hidden preference patterns such as genres, popularity, user taste, and movie characteristics.

8 Alternating Least Squares

Alternating Least Squares is another matrix factorization method. It alternately optimizes user factors and item factors while keeping the other fixed. ALS is commonly used for large-scale recommendation systems because it is efficient on sparse matrices.

9 Reason for Choosing SVD and ALS

User-based collaborative filtering was not selected because the Netflix dataset contains approximately 480,000 users, making full user-user similarity computation impractical. Item-based collaborative filtering was also not prioritized because matrix factorization captures item and user relationships implicitly. Therefore, SVD and ALS were selected as practical, scalable, and industry-relevant recommendation algorithms.

10 Problem Statement

The problem addressed in this project is personalized movie recommendation. Given a set of users, movies, and historical ratings, the goal is to recommend movies that a user has not yet watched but is likely to rate highly.

Let U be the set of users and M be the set of movies. The dataset contains observed ratings $r_{u,m}$ for some user-movie pairs. Since most users rate only a small number of movies, most entries in the user-movie matrix are missing.

The objective is to learn a function:

$$f(u, m) = \hat{r}_{u,m}$$

where:

- u is a user,
- m is a movie,
- $\hat{r}_{u,m}$ is the predicted rating.

After predicting ratings for unseen movies, the system ranks them and returns the Top-K recommendations.

11 Main Challenges

The key challenges are:

- The user-item matrix is highly sparse.
- The raw dataset is large and memory-intensive.
- Efficient preprocessing is required before model training.
- Models must generate recommendations for unseen movies.
- The recommendation system must be accessible through an interface.

12 Proposed Solution

The solution uses matrix factorization. Instead of comparing users directly, the model learns latent representations of users and movies. Each user is represented by a vector describing hidden preferences, and each movie is represented by a vector describing hidden movie attributes. The predicted rating is obtained from the interaction of these vectors.

The recommendation process follows these steps:

1. Load historical rating data.

- 2. Train a matrix factorization model.
- 3. Identify movies not rated by a target user.
- 4. Predict ratings for those unseen movies.
- 5. Sort movies by predicted rating.
- 6. Return the Top-K recommended movies.

13 Dataset Description and Preprocessing

14 Netflix Prize Dataset

The Netflix Prize Dataset is a benchmark dataset for recommendation systems. It contains user ratings for thousands of movies.

Table 1: Dataset Characteristics

Attribute	Description
Dataset	Netflix Prize Dataset
Users	Approximately 480,000
Movies	Approximately 17,700
Ratings	Over 100 million
Rating Scale	1 to 5
Data Type	User-Movie Rating Interactions

15 Raw Data Format

The raw Netflix dataset is stored in text files. Each movie ID appears as a separate header line, followed by multiple user ratings.

```
1 1:
2 1488844, 3, 2005-09-06
3 822109, 5, 2005-05-13
4 885013, 4, 2005-10-19
```

The above format is not directly suitable for model training. Therefore, preprocessing is required to convert it into a structured tabular format.

16 Processed Dataset Schema

After preprocessing, each record contains four columns.

Table 2: Processed Dataset Schema

Column	Description
user_id	Unique identifier of the user
movie_id	Unique identifier of the movie
rating	Rating given by the user
date	Date on which the rating was submitted

17 Why Polars and Parquet Were Used

The Netflix dataset is large, and loading it directly using traditional tools can consume significant memory. Polars was used because it provides fast and memory-efficient dataframe operations. The processed data was saved in Parquet format because Parquet is compressed, columnar, and faster to read for analytical workloads.

18 Sampling Strategy

To make development practical on laptop hardware, three dataset versions were planned:

Table 3: Sampling Strategy

Dataset Version	Purpose	Usage
1% Sample	Rapid debugging	Testing preprocessing and pipeline logic
10% Sample	Model training	SVD and ALS experiments
Full Dataset	Scalability testing	Optional final run

This approach allowed quick iteration while still preserving the ability to scale later.

19 Proposed Methodology

The proposed solution follows an end-to-end machine learning engineering workflow. The system was not designed as a single script but as a modular pipeline where each component performs a specific task.

20 Workflow Overview

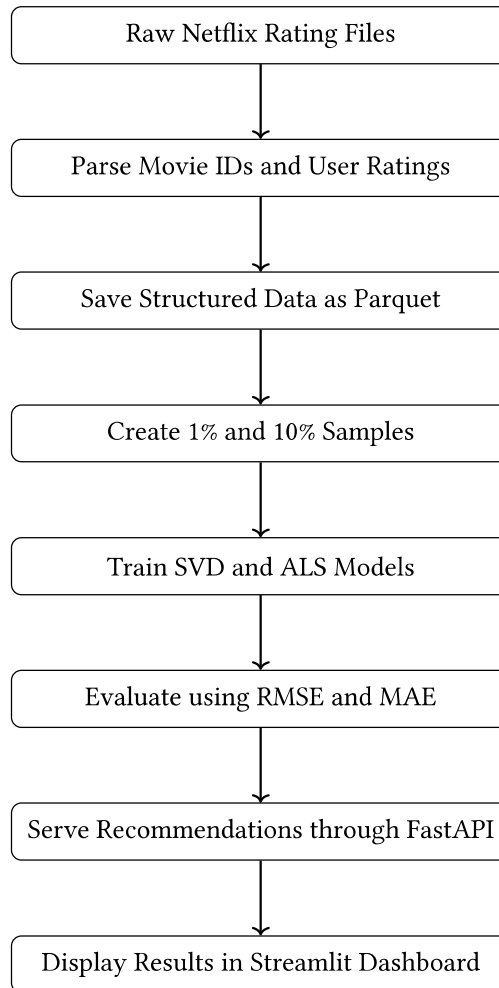


Figure 1: End-to-End Methodology

21 How the Recommendation Problem is Solved

The core idea is to transform sparse user-movie ratings into dense latent representations. The rating matrix is mostly empty because each user has rated only a few movies. Instead of filling the matrix manually, matrix factorization learns compact user and movie vectors.

For a user u and movie m , the predicted rating is computed as:

$$\hat{r}_{u,m} = p_u^T q_m$$

where:

- p_u is the latent vector of user u ,
- q_m is the latent vector of movie m ,

- $\hat{r}_{u,m}$ is the predicted rating.

Once predicted ratings are generated for all unseen movies, the system sorts them in descending order and returns the highest ranked movies.

22 Recommendation Generation

The recommendation engine works as follows:

1. Accept a user ID.
2. Retrieve the list of movies already rated by the user.
3. Remove already rated movies from the candidate set.
4. Predict ratings for all candidate movies.
5. Sort the predictions.
6. Return the Top-K recommendations.

23 System Architecture

24 Architecture Overview

The system architecture is divided into data, model, tracking, backend, and frontend layers.

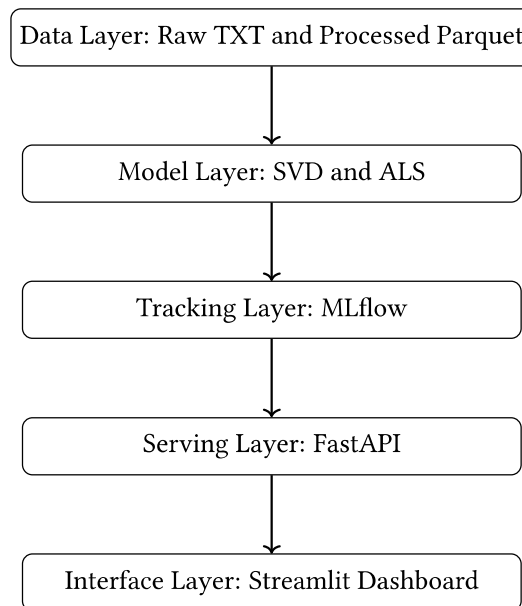


Figure 2: Layered System Architecture

25 Technology Stack

Table 4: Technology Stack

Component	Technology
Programming Language	Python
Data Processing	Polars, PyArrow
Modeling	Surprise, Implicit
Experiment Tracking	MLflow
Backend API	FastAPI
Frontend Dashboard	Streamlit
Model Storage	Pickle / Surprise Dump
Version Control	Git and GitHub

26 Model Development

27 SVD Model

Singular Value Decomposition was implemented using the Surprise library. The model learns latent factors for users and movies and predicts ratings for unseen user-movie pairs.

```
1 model = SVD(  
2     n_factors=50,  
3     n_epochs=20,  
4     lr_all=0.005,  
5     random_state=42  
6 )
```

The dataset was split into training and testing sets. The model was trained on the training set and evaluated on the test set.

28 ALS Model

ALS was implemented using the Implicit library. Before training ALS, user IDs and movie IDs were mapped to internal indices. A sparse user-item matrix was then created using SciPy's CSR matrix representation.

```
1 model = AlternatingLeastSquares(  
2     factors=64,  
3     regularization=0.1,  
4     iterations=15,  
5     random_state=42  
6 )
```

The ALS model was saved along with user and movie mapping dictionaries to ensure correct inference later.

29 Model Serialization

Trained models were serialized and stored as artifacts. This allows the backend service to load the trained model without retraining every time.

Artifacts include:

- SVD model binary
- ALS model binary
- User ID mapping
- Movie ID mapping

30 Experimental Results

31 Evaluation Metrics

The SVD model was evaluated using RMSE and MAE.

31.1 Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

RMSE gives higher penalty to large prediction errors.

31.2 Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

MAE represents the average absolute difference between actual and predicted ratings.

32 SVD Results

Table 5: SVD Model Performance

Metric	Value
RMSE	0.8483
MAE	0.6566

The RMSE value of 0.8483 shows that the model predicts ratings with strong accuracy for a baseline matrix factorization system.

33 Model Comparison

Table 6: Model Comparison

Model	RMSE	MAE	Purpose
SVD	0.8483	0.6566	Explicit rating prediction
ALS	–	–	Scalable matrix factorization

ALS was implemented mainly to demonstrate scalable matrix factorization and future model comparison. Ranking metrics such as Precision@K and NDCG@K can be added later for a fair comparison between SVD and ALS.

34 Experiment Tracking with MLflow

MLflow was integrated to track experiments, parameters, metrics, and artifacts.

35 Need for Experiment Tracking

Machine learning development involves multiple experiments with different hyperparameters, models, and datasets. Without experiment tracking, it becomes difficult to identify which configuration produced the best result. MLflow solves this problem by maintaining a record of all runs.

36 Tracked Parameters

The following parameters were logged:

- Algorithm name
- Number of latent factors
- Number of epochs
- Learning rate
- Number of ALS iterations
- Regularization value

37 Tracked Metrics

The following metrics were logged:

- RMSE
- MAE

- Training duration
- Matrix density

38 Tracked Artifacts

The trained models were logged as MLflow artifacts:

- `svd_model.pkl`
- `als_model.pkl`

MLflow improved reproducibility and made the project more suitable for real-world machine learning workflows.

39 Backend API and User Interface

40 FastAPI Backend

FastAPI was used to expose the trained recommendation model as a web service.

40.1 Health Check Endpoint

```
1 GET /
```

Example response:

```
1 {
2   "status": "ONLINE",
3   "service": "netflix-recsys-api"
4 }
```

40.2 Recommendation Endpoint

```
1 GET /recommend/{user_id}
```

This endpoint accepts a user ID and returns Top-K recommendations.

41 Streamlit Dashboard

Streamlit was used to build an interactive frontend. The dashboard allows users to enter a user ID, choose the number of recommendations, and view the recommended movies.

42 Frontend-Backend Flow

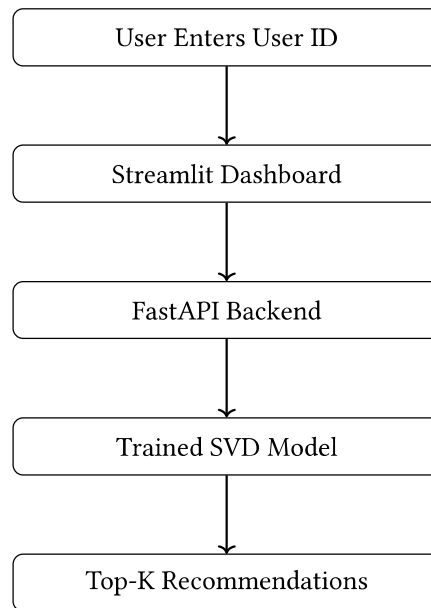


Figure 3: Recommendation Serving Workflow

43 System Validation

The final end-to-end system was tested by running FastAPI and Streamlit simultaneously. The dashboard successfully communicated with the backend and displayed recommendations generated by the trained model.

44 Results and Discussion

The project successfully implemented an end-to-end recommendation system. The most important achievement is that the system works beyond offline model training. It includes data engineering, modeling, experiment tracking, API serving, and dashboard visualization.

45 Completed Components

Table 7: Project Completion Status

Component	Status
Data Preprocessing	Completed
Parquet Conversion	Completed
Sampling Pipeline	Completed
SVD Model Training	Completed
ALS Model Training	Completed
RMSE and MAE Evaluation	Completed
MLflow Tracking	Completed
FastAPI Backend	Completed
Streamlit Dashboard	Completed
End-to-End Testing	Completed

46 Discussion

The SVD model achieved strong performance with an RMSE of 0.8483. This indicates that matrix factorization is effective for sparse movie rating data. The system was also designed in a modular way, allowing future improvements such as ranking metrics, movie title mapping, and public deployment.

The use of FastAPI and Streamlit makes the project demonstrable. This is important because recommendation systems are easier to understand when users can interact with them directly instead of only viewing model metrics.

47 Limitations and Future Work

48 Limitations

The current implementation has the following limitations:

- The dashboard currently displays movie IDs unless movie title mapping is added.
- ALS evaluation requires ranking-based metrics for proper comparison.
- Cold-start users and new movies are not fully handled.
- The system currently runs locally and is not yet deployed publicly.

49 Future Work

Future improvements include:

- Add movie title mapping using `movie_titles.csv`.
- Implement Precision@K, Recall@K, MAP@K, and NDCG@K.
- Add cold-start handling for new users.
- Dockerize the FastAPI and Streamlit services.
- Deploy the application using Streamlit Cloud, Render, or Railway.
- Add explainable recommendations such as “recommended because you liked similar movies”.
- Build a hybrid recommendation system using movie metadata.

50 Conclusion

This project successfully developed a Personalized Content Discovery and Recommendation System using the Netflix Prize Dataset. The system applies matrix factorization techniques to learn hidden user preferences and movie characteristics from historical ratings.

The SVD model achieved an RMSE of 0.8483 and an MAE of 0.6566, showing strong performance for rating prediction. ALS was also implemented as a scalable matrix factorization approach. The integration of MLflow, FastAPI, and Streamlit transformed the project into a complete machine learning engineering system rather than a simple model training notebook.

Overall, the project demonstrates practical skills in recommendation systems, data preprocessing, model development, experiment tracking, backend API design, and dashboard development. These skills are directly relevant to machine learning, data science, and software engineering roles.

References

- [1] Netflix Prize Dataset.
- [2] Surprise Recommendation System Library Documentation.
- [3] Implicit Library Documentation.
- [4] FastAPI Documentation.
- [5] Streamlit Documentation.
- [6] MLflow Documentation.
- [7] Y. Koren, R. Bell, and C. Volinsky, “Matrix Factorization Techniques for Recommender Systems,” *IEEE Computer*, 2009.
- [8] F. Ricci, L. Rokach, and B. Shapira, *Recommender Systems Handbook*.